

Relatório de Evolução — Sprint 03

EV Challenge — GoodWe

Prompt and Artificial Intelligence — Ciência da Computação — FIAP

1. Resumo da evolução

Nas Sprints 1 e 2 foi desenvolvido o GoodWezinho, chatbot da GoodWe voltado para usuários de veículos elétricos e interessados nos carregadores da linha HCA G2.

Na Sprint 1, o projeto possuía um fluxo conversacional controlado manualmente, um banco de veículos elétricos e funções para auxiliar no cálculo de custo e tempo de carregamento.

Na Sprint 2, a solução foi ampliada com uma arquitetura RAG, permitindo utilizar o datasheet oficial do carregador HCA G2 como base de conhecimento.

Na Sprint 03, o núcleo conversacional foi refatorado com LangGraph. A nova arquitetura passou a utilizar memória por sessão, rotas específicas de execução, guardrails de segurança e comparação sistemática entre diferentes modelos de linguagem.

2. Refatoração da arquitetura

O framework escolhido foi o LangGraph.

A escolha ocorreu porque o framework permite controlar explicitamente o fluxo do agente através de estados, nós e rotas condicionais, além de possuir suporte nativo para memória conversacional.

A arquitetura da Sprint 03 utiliza:

- StateGraph para controlar o fluxo;
- MessagesState para manter as mensagens;
- InMemorySaver para memória por sessão;
- thread_id para separar diferentes conversas;
- nós específicos para respostas, cálculos e guardrails;
- RAG com o datasheet oficial do HCA G2;
- banco de veículos herdado das Sprints anteriores.

O fluxo passou a decidir qual ação deve ser executada de acordo com a mensagem do usuário. Solicitações de preço e tempo de carregamento são direcionadas para funções específicas, enquanto perguntas gerais utilizam a LLM com o contexto recuperado pelo RAG.

Tentativas de Prompt Injection e solicitações elétricas perigosas podem ser bloqueadas antes mesmo de uma chamada ao modelo.

Trade-offs

A nova arquitetura ficou mais organizada e controlável, porém possui mais componentes do que a implementação anterior.

Também existe maior dependência do framework e das bibliotecas utilizadas no projeto.

Por outro lado, essa complexidade permitiu separar responsabilidades, implementar memória por sessão e criar mecanismos de segurança mais claros.

3. Comparativo antes × depois

Característica	Sprints 1 e 2	Sprint 03
Arquitetura	Fluxo controlado manualmente	LangGraph
Modelo	GPT-4o-mini	Gemini 3.5 Flash-Lite
Histórico	Lista de mensagens controlada pelo código	Memória do framework
Sessões	Histórico único/manual	thread_id

Característica	Sprints 1 e 2	Sprint 03
Memória	Implementação manual	InMemorySaver
RAG	Similarity search + PDF HCA G2	RAG integrado ao agente
Banco de veículos	Sim	Mantido
Cálculo de recarga	Sim	Mantido em rota específica
Prompt Injection	Sem guardrail dedicado	Guardrail no fluxo
Segurança elétrica	Principalmente via prompt	Nó específico de bloqueio
Testes de modelos	Modelo definido previamente	Flash × Flash-Lite
Avaliação	Testes anteriores	Funcional, memória, segurança, latência e tokens

Resultados da nova arquitetura

Nos testes realizados, os dois modelos avaliados conseguiram:

- utilizar corretamente o RAG;
- manter informações durante a mesma sessão;
- resistir ao teste de Prompt Injection;
- evitar orientações elétricas perigosas;
- recusar aconselhamento jurídico e financeiro inadequado;
- evitar a invenção de especificações técnicas.

A memória foi demonstrada em três turnos utilizando um BYD Dolphin Mini.

O usuário informou o veículo apenas no primeiro turno e o agente conseguiu utilizar essa informação posteriormente para calcular:

- custo aproximado da carga completa: R\$ 19,40;
- tempo estimado utilizando 22 kW: 1 hora e 46 minutos.

Na comparação entre modelos:

Modelo	Latência média	Tokens registrados
Gemini 3.5 Flash	9,09 s	9.771
Gemini 3.5 Flash-Lite	0,81 s	7.275

Como ambos apresentaram comportamento adequado nos testes, o Gemini 3.5 Flash-Lite foi escolhido para a versão final por apresentar menor latência e menor utilização de tokens nos testes registrados.

4. Problemas encontrados e soluções

Problema 1 — Armazenamento da API Key no Google Colab

Durante o desenvolvimento, o Google Colab apresentou erro ao tentar salvar a chave da API através da funcionalidade Secrets.

Alternativas consideradas:

- utilizar o Colab Secrets;
- escrever a chave diretamente no código;
- carregar a chave somente durante a execução.

A chave não foi colocada diretamente no código para evitar exposição de credenciais.

A solução adotada foi solicitar a chave durante a execução e armazená-la temporariamente em uma variável de ambiente.

Essa solução manteve a credencial fora dos arquivos do projeto e do histórico do Git.

Problema 2 — Atualização dos módulos no Colab

Durante os testes, uma nova versão do arquivo `agent.py` foi salva, mas o Colab continuou utilizando a versão anteriormente importada.

Isso fez com que o agente ignorasse inicialmente as novas rotas de cálculo e enviasse algumas perguntas diretamente para a LLM.

Alternativas consideradas:

- reiniciar completamente o ambiente de execução;
- executar novamente todas as células;
- recarregar apenas os módulos alterados.

A solução adotada foi utilizar `importlib.reload()` para atualizar os módulos sem precisar reiniciar todo o ambiente.

Após isso, as rotas de preço e tempo funcionaram corretamente.

5. Divisão da equipe

O projeto EV Challenge é desenvolvido pelo grupo em diferentes disciplinas. Para organizar as entregas, os integrantes dividiram as responsabilidades entre as matérias do projeto.

Nesta Sprint da disciplina de Prompt and Artificial Intelligence, João Augusto Poloniato Telles ficou como principal responsável pelo desenvolvimento e organização da entrega.

Integrante	RM	Responsabilidade
Guilherme Figueira Velloso	568827	Integrante do grupo multidisciplinar do EV Challenge
José Augusto Ribeiro Freire Manfrinato	571151	Integrante do grupo multidisciplinar do EV Challenge
Lais da Silva Dias	569943	Integrante do grupo multidisciplinar do EV Challenge
João Augusto Poloniato Telles	571443	Responsável principal pela Sprint de Prompt and Artificial Intelligence
Thiago Soalheiro Diamantino	569316	Integrante do grupo multidisciplinar do EV Challenge
Kauan Damasceno de Lima	573727	Integrante do grupo multidisciplinar do EV Challenge

Conclusão

A Sprint 03 representou uma evolução da arquitetura do GoodWezinho.

As funcionalidades desenvolvidas nas etapas anteriores foram mantidas, mas passaram a operar dentro de uma arquitetura baseada em agentes, com memória por sessão, RAG integrado, guardrails e avaliação sistemática dos modelos.

Os testes demonstraram que a nova arquitetura tornou o fluxo mais controlável e permitiu incorporar mecanismos de segurança e memória sem eliminar as funcionalidades existentes.